



UNIVERSITAT
Carlemany

Formación europea
para desafíos reales

De:

 Planeta Formación y Universidades

Bloque 2- Introducción a patrones arquitectónicos

Aplicaciones basadas en tecnologías web

Bloque 2 — Introducción a patrones arquitectónicos

- Fundamentos de Patrones
- CRUD como Punto de Partida
- Patrones de Acceso a Datos
- Separación de Lecturas y Escrituras (CQRS)
- Ejemplos Guiados
- More Patterns
- Challenge FizzBuzz in C#

1. Fundamentos de Patrones

- Qué es un Patrón de Diseño (y por qué existe)
- Clasificación de Patrones: Creacionales, Estructurales y de Comportamiento

Qué es un Patrón de Diseño (y por qué existe)

Un **patrón de diseño** es una **solución conceptual reutilizable** a un problema que aparece de forma recurrente en el diseño de software.

Un patrón:

- **no es código**
- **no es una librería**
- **no es una clase concreta**
- es una **estructura de solución** que se puede adaptar a distintos lenguajes y tecnologías

Los patrones surgen porque:

- los sistemas grandes repiten los mismos problemas
- ciertas soluciones han demostrado funcionar mejor que otras
- permiten comunicar decisiones de diseño con un lenguaje común

Ejemplo conceptual:

Decir “uso Repository Pattern” explica **más** que mostrar 20 líneas de código de acceso a datos.

 *Los patrones no garantizan que el sistema sea correcto, pero reducen la probabilidad de errores de diseño.*

Clasificación de Patrones: Creacionales, Estructurales y de Comportamiento

Los patrones se agrupan según **el tipo de problema que resuelven**:

◆ Patrones Creacionales

Se centran en **cómo se crean los objetos**:

- controlan la instanciación
- reducen el acoplamiento
- evitan dependencias rígidas

Ejemplos: Singleton, Factory Method

◆ Patrones Estructurales

Se centran en **cómo se organizan y conectan las piezas**:

- encapsulan subsistemas
- separan responsabilidades
- simplifican dependencias

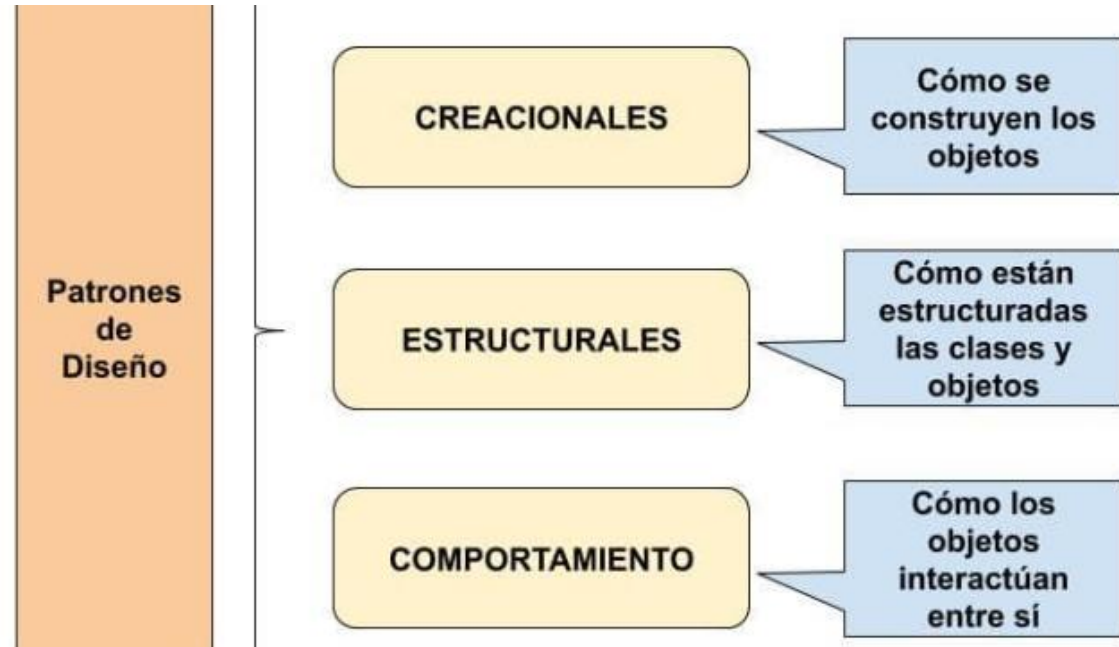
Ejemplos: Facade, Adapter, Repository

◆ Patrones de Comportamiento

Se centran en **cómo interactúan los componentes**:

- flujo de información
- reparto de responsabilidades
- coordinación entre objetos

Ejemplos: Observer, Strategy, Command



 *Un sistema real suele combinar patrones de los tres tipos.*

2.CRUD como Punto de Partida

- CRUD como Patrón Básico de Aplicación
- CRUD y su Relación con APIs REST
- Más Allá de CRUD: Cuando Aparecen Nuevos Patrones

CRUD como Patrón Básico de Aplicación

CRUD representa el **modelo más básico de interacción con datos** y aparece en la mayoría de aplicaciones de negocio.

CRUD define cuatro operaciones fundamentales:

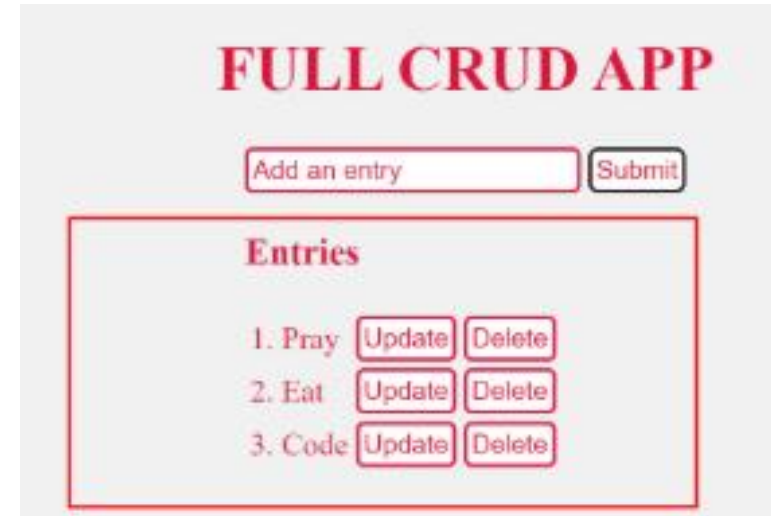
- **Create** → crear una entidad
- **Read** → consultar una entidad
- **Update** → modificar una entidad existente
- **Delete** → eliminar una entidad

Este patrón es tan común porque:

- refleja directamente el modelo de datos
- encaja con bases de datos relacionales
- es fácil de entender y de implementar

Limitación clave:

- lectura y escritura **comparten el mismo modelo**
- no distingue entre operaciones simples y consultas complejas



 *CRUD es un buen punto de partida, pero no siempre es suficiente para sistemas reales.*

CRUD y su Relación con APIs REST

En APIs REST, el patrón CRUD suele expresarse mediante:

- **recursos** (por ejemplo /assets)
- **métodos HTTP** que representan las operaciones

Correspondencia típica:

- POST /assets → Create
- GET /assets/{id} → Read
- PUT /assets/{id} → Update
- DELETE /assets/{id} → Delete

Ventajas:

- claridad semántica
- contratos fáciles de entender
- interoperabilidad entre sistemas

Problema habitual:

- forzar cualquier operación a ser CRUD
- usar GET para cálculos complejos o agregados

 *No todo endpoint debe ser CRUD, aunque el sistema tenga CRUD.*

```
namespace Fleet_Managment.Controllers
{
    [Route("api/[controller]")]
    0 başvuru
    public class UsersController : ControllerBase
    {
        private List<User> _users = FakeData.GetUsers(200);

        [HttpGet]
        0 başvuru
        public List<User> Get()
        {
            return _users;
        }
    }
}
```

```
public class HomeController : Controller
{
    [HttpGet]
    public string Index()
    {
        return "GET METHOD";
    }

    [HttpPut]
    public string Put()
    {
        return "PUT METHOD";
    }

    [HttpDelete]
    public string Delete()
    {
        return "DELETE METHOD";
    }
}
```


! Más Allá de CRUD: Cuando Aparecen Nuevos Patrones

En sistemas reales, aparecen necesidades que CRUD no cubre bien:

- cálculos agregados
- reglas de negocio complejas
- transacciones con múltiples entidades
- separación entre lógica y persistencia

Ejemplo claro:

- gestionar activos → CRUD
- calcular exposición total → **no es CRUD**

Esto obliga a introducir:

- capas de acceso a datos más estructuradas
- patrones que separen responsabilidades
- modelos más expresivos que “leer/escribir filas”

 *Aquí es donde entran Repository Pattern, Unit of Work y CQRS.*

3. Patrones de Acceso a Datos

- Repository Pattern — Problema que Resuelve
- Repository Pattern — Estructura Conceptual
- Unit of Work — Motivación y Problema de Fondo
- Unit of Work — Funcionamiento Conceptual
- Comparativa: Sin Repository vs Repository + Unit of Work
- Repository + Unit of Work en Conjunto (Visión Arquitectónica)



Repository Pattern — Problema que Resuelve

En aplicaciones reales, uno de los principales problemas de diseño es **mezclar la lógica de negocio con el acceso a datos**.

Sin Repository:

- el controlador conoce SQL, ORM o detalles de persistencia
- la lógica de negocio depende directamente de la base de datos
- los cambios en persistencia se propagan por todo el sistema

El **Repository Pattern** surge para resolver esto:

- encapsula el acceso a datos
- presenta los datos como una **colección de objetos de dominio**
- oculta cómo y dónde se almacenan



El repositorio no es una base de datos: es una abstracción semántica del dominio.

Repository Pattern — Estructura Conceptual

Conceptualmente, un repositorio:

- define una **interfaz** clara
- expone operaciones significativas del dominio
- evita que el resto del sistema “piense en tablas”

Operaciones típicas:

- añadir una entidad
- recuperar una entidad por id
- actualizar o eliminar entidades
- ejecutar consultas propias del dominio

Ejemplo conceptual:

- Add(Asset)
- GetById(assetId)
- Remove(asset)
- GetTotalExposure()

 *Las consultas del repositorio deben expresar intención de negocio, no detalles técnicos.*

Unit of Work — Motivación y Problema de Fondo

En sistemas con varias operaciones relacionadas:

- no basta con guardar entidades de forma aislada
- se necesita **consistencia global**

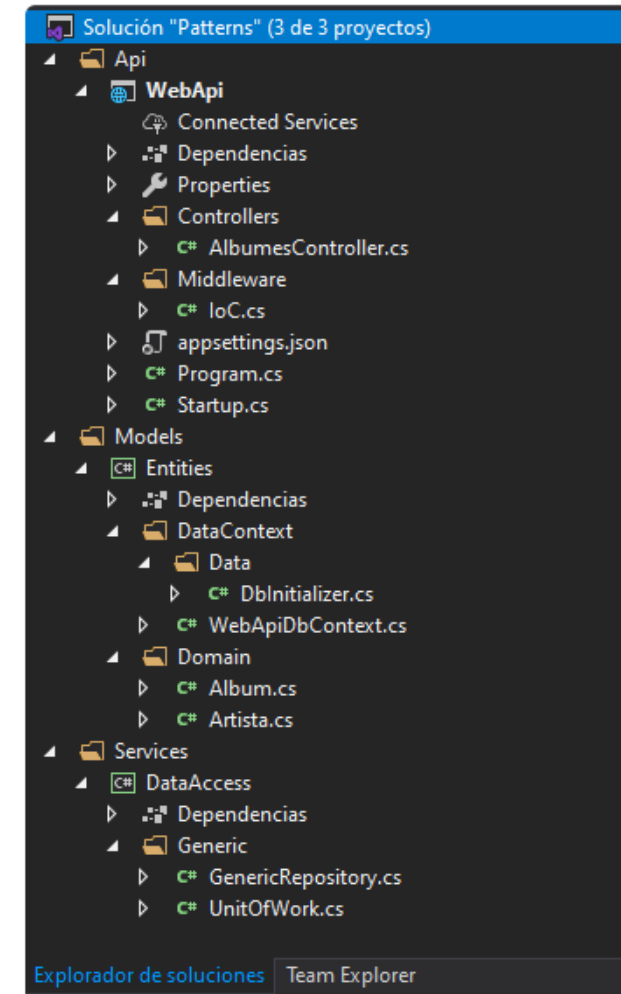
El problema:

- varias entidades cambian como parte de una misma operación
- si una falla, el sistema puede quedar inconsistente

El **Unit of Work** aparece para:

- agrupar cambios
- controlar cuándo se persisten
- asegurar atomicidad conceptual

 *Unit of Work representa una transacción lógica del dominio.*



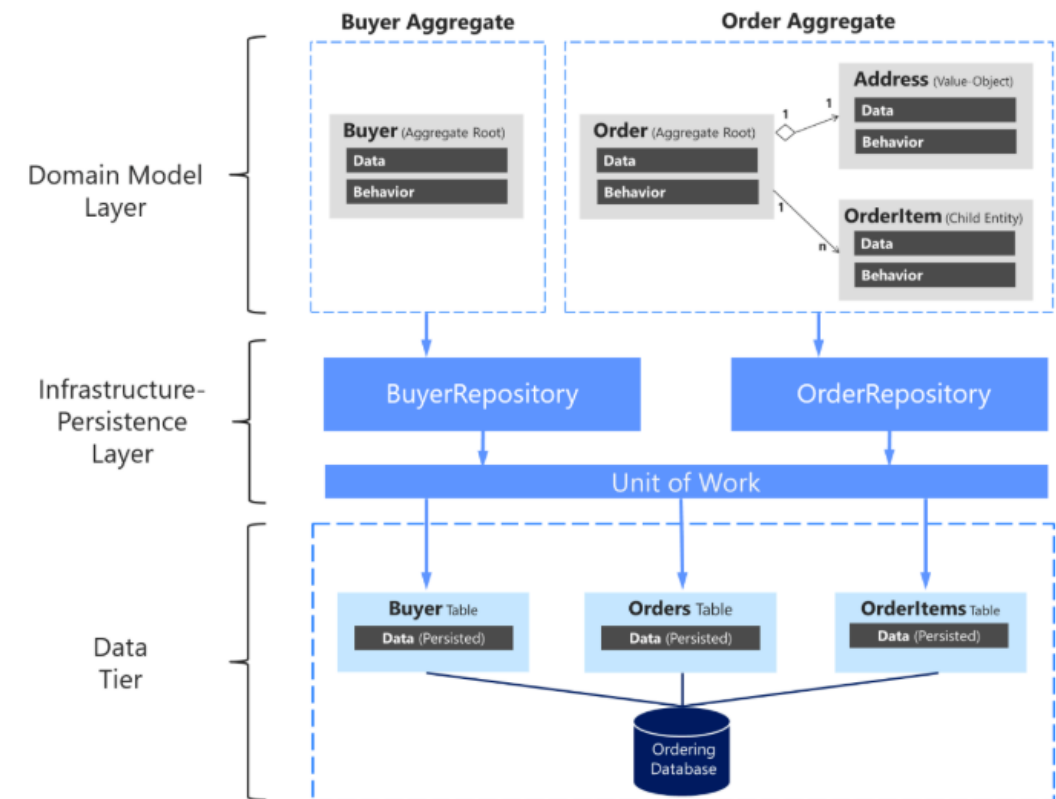
Unit of Work — Funcionamiento Conceptual

Conceptualmente, una Unit of Work:

- mantiene un registro de entidades nuevas, modificadas o eliminadas
- coordina la escritura de todos los cambios
- expone un punto único de confirmación (Commit)

Flujo mental típico:

1. el controlador inicia una operación
2. interactúa con uno o varios repositorios
3. la Unit of Work acumula los cambios
4. se confirma la operación como un todo



 Aunque la implementación sea simple o dummy, el concepto debe estar presente en el diseño.

Comparativa: Sin Repository vs Repository + Unit of Work

Sin Repository / Unit of Work

- Controladores acceden directamente a datos
- Lógica de negocio mezclada con persistencia
- Código más acoplado y difícil de testear

 *Funciona, pero no escala bien.*

Con Repository + Unit of Work

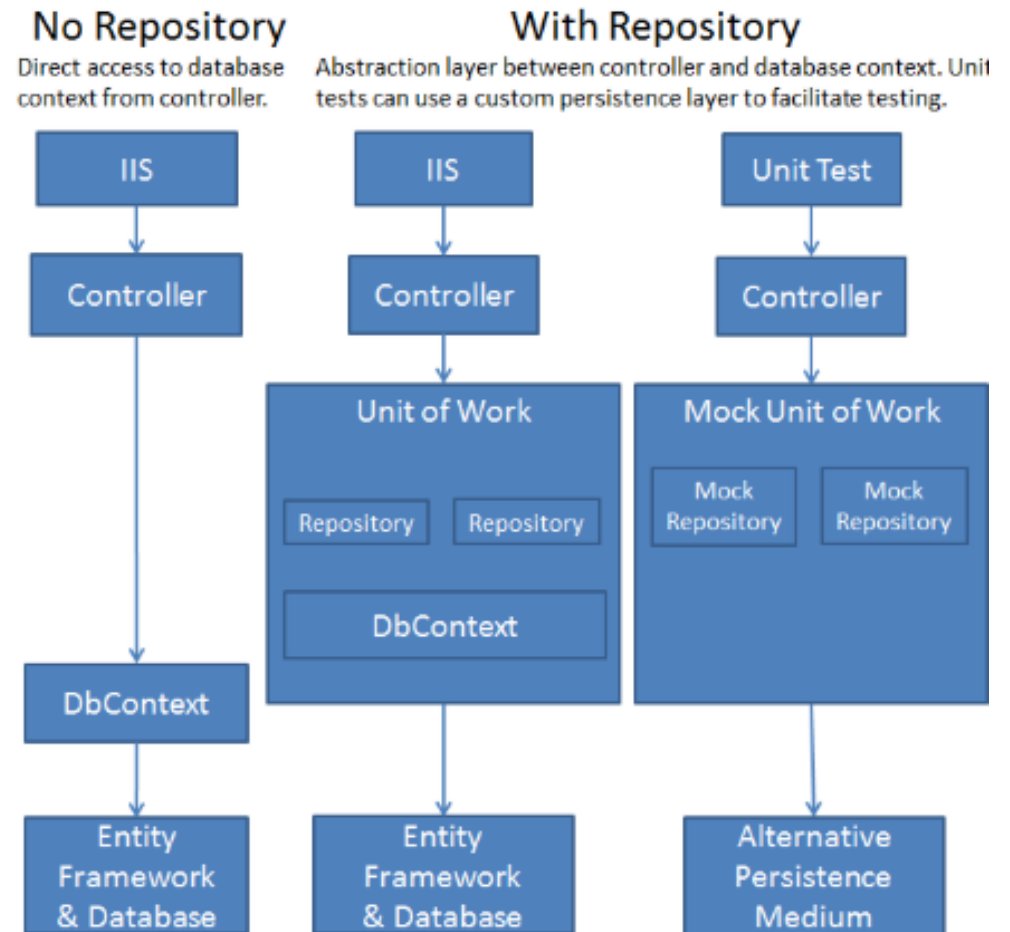
- Acceso a datos encapsulado
- Responsabilidades bien separadas
- Operaciones coordinadas y testeables

 *Más estructura, más mantenibilidad.*

Idea clave

Repository abstrae los datos.
Unit of Work coordina los cambios.

 *La aplicación funciona, pero la arquitectura no escala bien.*



Repository + Unit of Work en Conjunto (Visión Arquitectónica)

Repository y Unit of Work **no son patrones aislados**, están pensados para trabajar juntos.

- El **Repository** define *qué* operaciones de datos existen
- La **Unit of Work** controla *cuándo* se aplican los cambios

En conjunto permiten:

- desacoplar la lógica de negocio de la persistencia
- coordinar múltiples operaciones como una sola unidad lógica
- simplificar el flujo de los controladores

 *El controlador orquesta el caso de uso, pero no gestiona datos directamente.*

4. Separación de Lecturas y Escrituras (CQRS)

- CQRS — Origen y Principio Conceptual
- CQRS — Fundamento Teórico (Command–Query Responsibility Segregation)
- Por Qué CRUD se Queda Corto y Aparece CQRS
- CQRS en la Práctica: Queries de Dominio

CQRS — Origen y Principio Conceptual

CQRS (Command Query Responsibility Segregation) parte de una idea simple:

- **Commands** → cambian el estado del sistema
- **Queries** → leen información, sin efectos colaterales

Este principio rompe con el enfoque CRUD clásico:

- no todo es crear/leer/actualizar/eliminar
- algunas operaciones representan **consultas de negocio**

 *CQRS no elimina CRUD, lo complementa cuando el dominio lo requiere.*

CQRS — Fundamento Teórico (Command–Query Responsibility Segregation)

CQRS es un patrón arquitectónico que propone una **separación estricta** entre:

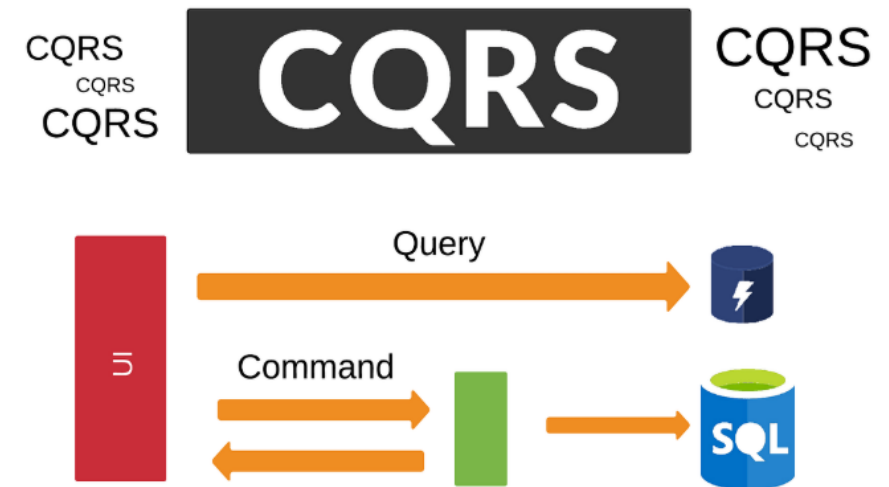
- **Commands**: operaciones que **cambian el estado** del sistema
- **Queries**: operaciones que **leen información**, sin efectos colaterales

Este principio deriva de una idea más básica:

Una consulta no debería modificar el estado del sistema.

En CQRS:

- escribir y leer **no son simétricos**
- tienen **objetivos distintos**
- y, por tanto, **merecen modelos distintos**



 *CQRS no es una tecnología, es una forma distinta de pensar el diseño del sistema.*

🧠 Por Qué CRUD se Queda Corto y Aparece CQRS

El enfoque CRUD asume que:

- el mismo modelo sirve para leer y escribir
- todas las operaciones son equivalentes
- los datos se usan igual en ambos sentidos

En sistemas reales esto no suele cumplirse:

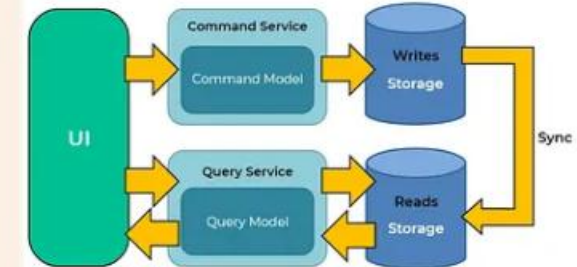
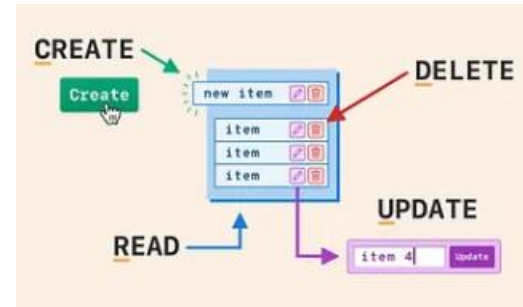
- las **escrituras** requieren validaciones, reglas y consistencia
- las **lecturas** requieren agregaciones, proyecciones y rendimiento

Consecuencia:

- modelos inflados
- código difícil de mantener
- consultas complejas forzadas en estructuras CRUD

CQRS surge para:

- liberar a las queries de las restricciones del modelo de escritura
- permitir modelos optimizados para lectura
- expresar mejor la intención de cada operación



 *CQRS separa complejidad en lugar de acumularla.*



CQRS en la Práctica: Queries de Dominio

Una **Query** en CQRS:

- no es “leer una entidad”
- es **responder una pregunta del dominio**

Ejemplos típicos:

- “¿Cuál es la exposición total del sistema?”
- “¿Qué activos superan cierto umbral?”
- “¿Cuál es el riesgo agregado por zona?”

Características de una Query:

- no modifica datos
- puede usar modelos distintos
- puede optimizarse sin afectar a escrituras

En la tarea:

- GET /assets/{id} → lectura CRUD
- GET /assets/total-exposure → **Query CQRS**

 *No todo GET es CRUD; algunas lecturas son queries de dominio.*

5.Ejemplos Guiados

- Ejemplo 1 — Repository + Unit of Work
- Ejemplo 2 — CQRS Ligero (Commands vs Queries en la API)

Ejemplo 1 — Repository + Unit of Work

Idea clave para el alumno

- El **controller** nunca accede a datos directamente
- Todo pasa por **UoW** → **Repository**
- Cambiar InMemory por SQL **no cambia los endpoints**

```
// 4. Implementación mínima (InMemory o SQL)
public class UnitOfWork : IUnitOfWork {
    public IAssetRepository Assets { get; } = new AssetRepository();
    public void Commit() { /* SaveChanges() o no-op */ }
}
```

```
// 1. Entidad del dominio
public class Asset {
    public Guid Id { get; set; }
    public decimal AssetValue { get; set; }
}

// 2. Repository: TODO acceso a datos pasa por aquí
public interface IAssetRepository {
    Asset? GetById(Guid id);
    void Add(Asset asset);
    void Update(Asset asset);
    void Delete(Guid id);

    decimal GetTotalExposure(); // consulta de dominio
}

// 3. Unit of Work: agrupa repositorios + commit
public interface IUnitOfWork {
    IAssetRepository Assets { get; }
    void Commit();
}
```

Ejemplo 2 — CQRS Ligero (Commands vs Queries en la API)

Idea clave para el alumno

- **Commands** → modifican datos → Commit()
- **Queries** → solo leen → NO Commit()
- total-exposure es CQRS porque **responde una pregunta**, no devuelve una entidad

```
// COMMANDS (cambian estado)
POST /assets
PUT  /assets/{id}
DELETE /assets/{id}

// QUERY CRUD (Leer una entidad)
GET /assets/{id}

// QUERY DE DOMINIO (no CRUD)
GET /assets/total-exposure
```

```
// Ejemplo mental de implementación
CreateAsset(asset):
    uow.Assets.Add(asset)
    uow.Commit()

GetTotalExposure():
    return uow.Assets.GetTotalExposure()
```


6. More Patterns

- Factory Pattern (Creacional)
- Singleton Pattern (Creacional)
- Rules Engine / Rule Machine Pattern (Comportamiento)
- Strategy Pattern (Comportamiento)



Factory Pattern (Creacional)

El **Factory Pattern** es un patrón creacional cuyo objetivo es **desacoplar la creación de objetos de su uso**.

En lugar de que una clase decida **cómo** crear un objeto concreto:

- delega esa responsabilidad en una fábrica
- trabaja contra **interfaces o abstracciones**

Esto permite:

- cambiar implementaciones sin modificar el código cliente
- centralizar la lógica de creación
- evitar dependencias rígidas con clases concretas

Ejemplo conceptual:

- el sistema necesita un repositorio
- el código **no decide** si es SQL, InMemory o Mock
- la fábrica devuelve la implementación adecuada



El Factory Pattern responde a “qué necesito”, no a “cómo se construye”.

Singleton Pattern (Creacional)

El **Singleton Pattern** garantiza que:

- una clase tenga **una única instancia**
- y proporciona un punto de acceso global a ella

Se utiliza cuando:

- existe un recurso compartido
- el estado debe ser único
- crear múltiples instancias sería problemático

Ejemplos típicos:

- gestor de configuración
- gestor de conexiones
- cache global

 Riesgos importantes:

- estado global difícil de testear
- dependencias ocultas
- abuso del patrón

 *Singleton debe usarse con criterio: no todo lo “global” debe ser Singleton.*

Rules Engine / Rule Machine Pattern (Comportamiento)

El **Rules Engine Pattern** separa las **reglas de negocio** de la lógica principal de la aplicación.

En lugar de:

- grandes bloques de if/else
- reglas mezcladas con el flujo principal

Se introduce:

- un conjunto de **reglas independientes**
- un **motor** que las evalúa
- un **contexto de datos** común

Ventajas:

- reglas fáciles de modificar
- sistema extensible sin tocar el núcleo
- mejora la mantenibilidad

Ejemplo conceptual:

- reglas de riesgo
- reglas de validación
- reglas de decisión

 *Las reglas cambian más rápido que el código: sepáralas.*

Strategy Pattern (Comportamiento)

El **Strategy Pattern** permite definir una **familia de algoritmos**, encapsularlos y hacerlos intercambiables.

En lugar de:

- condicionales para elegir comportamiento
- lógica rígida embebida en una clase

Se define:

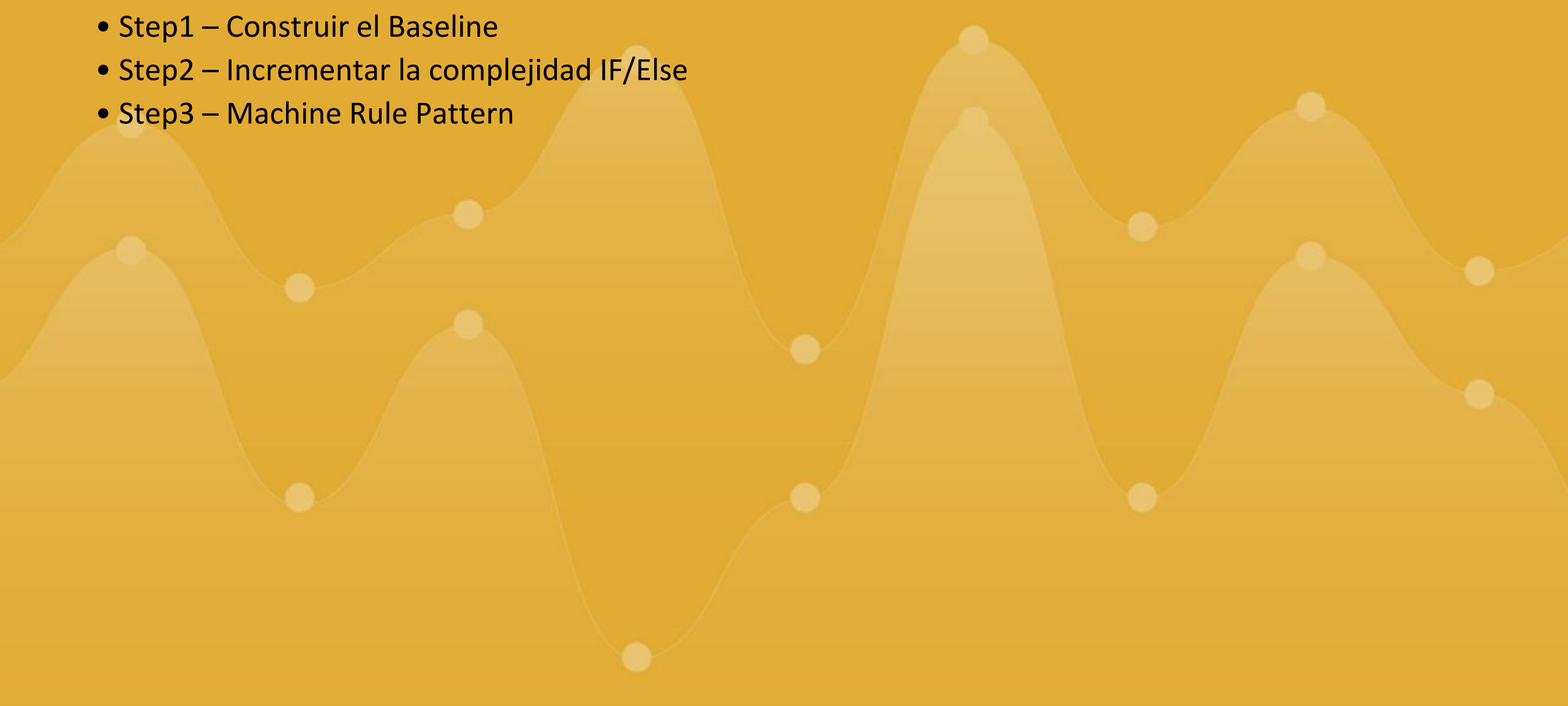
- una interfaz común
- múltiples estrategias concretas
- selección dinámica de la estrategia

Ejemplos claros:

- distintos cálculos de exposición
- diferentes políticas de validación
- múltiples formas de procesar datos

 *Strategy permite cambiar el “cómo” sin cambiar el “quién lo usa”.*

7. Challenge FizzBuzz in C#

- Step1 – Construir el Baseline
 - Step2 – Incrementar la complejidad IF/Else
 - Step3 – Machine Rule Pattern
- 

Challenge - FizzBuzz

Vamos a resolver un problema simple (FizzBuzz) y luego lo iremos complicando.

El objetivo no es “FizzBuzz”, sino aprender a **mejorar diseño** cuando aparecen requisitos nuevos:

- primero funciona 
- luego se vuelve difícil de mantener 
- y finalmente aplicamos un patrón (Rule Machine)  

 Cada paso introduce una necesidad que hace que el diseño anterior “duela”.

1 Step 1: FizzBuzz (Enunciado)

Escribe código que imprima, para un rango contiguo de números:

- el número, si no cumple ninguna regla
- "fizz" si es múltiplo de 3
- "buzz" si es múltiplo de 5
- "fizzbuzz" si es múltiplo de 15

Ejemplo (1–20):

1 2 fizz 4 buzz fizz 7 8 fizz buzz 11 fizz 13 14 fizzbuzz 16 17 fizz 19 buzz



Además: **añade Unit Tests** que prueben la lógica.

Step 1: Idea de solución (baseline con if)

Estrategia inicial (la más directa):

- recorrer el rango (for)
- para cada número, decidir qué imprimir con if/else
- separar la lógica en un método Render(int n) para poder testear

 Aunque lo ejecutes desde Main, el truco docente es aislar la lógica en un método para tests.

Program.cs

```
csharp Copia el codi

using System;

namespace FizzBuzzStep1
{
    class Program
    {
        static void Main(string[] args)
        {
            FizzBuzz fizzBuzz = new FizzBuzz();
            fizzBuzz.SequenceFizzBuzz(100);

            Console.WriteLine("Press ENTER to close console.....");
            string line = Console.ReadLine();

            if (line == "enter")
            {
                System.Environment.Exit(0);
            }
        }
    }
}
```

Step 1: Idea de solución (baseline con if) – Parte 1

Estrategia inicial (la más directa):

- recorrer el rango (for)
- para cada número, decidir qué imprimir con if/else
- separar la lógica en un método Render(int n) para poder testear

 Aunque lo ejecutes desde Main, el truco docente es aislar la lógica en un método para tests.

Program.cs

```
csharp Copia el codi

using System;

namespace FizzBuzzStep1
{
    class Program
    {
        static void Main(string[] args)
        {
            FizzBuzz fizzBuzz = new FizzBuzz();
            fizzBuzz.SequenceFizzBuzz(100);

            Console.WriteLine("Press ENTER to close console.....");
            string line = Console.ReadLine();

            if (line == "enter")
            {
                System.Environment.Exit(0);
            }
        }
    }
}
```

Step 1: Idea de solución (baseline con if) – Parte 2

```
/// <summary>
/// Print the sequence FizzBuzz.
/// </summary>
/// <param name="endSequence">
/// An int containing the end of sequence about FizzBuzz.
/// </param>
public void SequenceFizzBuzz(int endSequence)
{
    for (int i = 1; i <= endSequence; i++)
    {
        string sequenceFizzBuzz = ComputationOutputIterationFizzBuzz(i);
        Console.WriteLine(sequenceFizzBuzz + " ");
    }
}
```

```
/// <summary>
/// Returns the corresponding value of the iteration executed.
/// </summary>
/// <param name="iteration">
/// An int containing the corresponding iteration value about FizzBuzz.
/// </param>
/// <returns>The iteration computation value.</returns>
public string ComputationOutputIterationFizzBuzz(int iteration)
{
    bool canBeMultipliedByFive = CanBeMultipliedBy(iteration, 5);
    bool canBeMultipliedByThree = CanBeMultipliedBy(iteration, 3);
    string sequenceFizzBuzz = "";

    if (canBeMultipliedByThree)
    {
        sequenceFizzBuzz += "Fizz";
    }

    if (canBeMultipliedByFive)
    {
        sequenceFizzBuzz += "Buzz";
    }

    if (string.IsNullOrEmpty(sequenceFizzBuzz))
    {
        sequenceFizzBuzz = iteration.ToString();
    }

    return sequenceFizzBuzz;
}
```

```
/// <summary>
/// Returns true if sourceNumber is module of targetNumber.
/// </summary>
/// <param name="sourceNumber">
/// An int containing the number that wants to do the module.
/// </param>
/// <param name="targetNumber">
/// An int containing the number that applies the module.
/// </param>
/// <returns>
/// True if sourceNumber is module of targetNumber or false if sourceNumber
/// is not module of targetNumber.
/// </returns>
private bool CanBeMultipliedBy(int sourceNumber, int targetNumber)
{
    return (sourceNumber % targetNumber) == 0;
}
```

Step 1: Qué hemos conseguido (y qué NO)– Parte 3



- solución correcta
- lógica fácilmente testeable (Render aislado)
- tests prueban reglas clave

Pero...

- la lógica está “codificada” con if/else
- si aparecen nuevas reglas, el método crecerá
- el orden de condiciones se vuelve delicado (15 antes que 3/5)

```
namespace FizzBuzzStep1.Tests
{
    public class FizzBuzzTests
    {
        [Theory]
        [InlineData(1, "1")]
        [InlineData(2, "2")]
        [InlineData(3, "Fizz")]
        [InlineData(4, "4")]
        [InlineData(5, "Buzz")]
        [InlineData(6, "Fizz")]
        [InlineData(9, "Fizz")]
        [InlineData(10, "Buzz")]
        [InlineData(15, "FizzBuzz")]
        [InlineData(30, "FizzBuzz")]
        public void ComputationOutputIterationFizzBuzz_ReturnsExpected(int input, string expected)
        {
            // Arrange
            var sut = new FizzBuzz();

            // Act
            var result = sut.ComputationOutputIterationFizzBuzz(input);

            // Assert
            Assert.Equal(expected, result);
        }
    }
}
```

2 Slide — Step 2: Fizz.Buz (Requisito nuevo)

Añade una regla nueva:

- Si el número **contiene un 3**, debe imprimirse **"lucky"**
- Esta regla **sobrescribe cualquier comportamiento anterior** (prioridad máxima)

Ejemplo (1–20):

1 2 lucky 4 buzz fizz 7 8 fizz buzz 11 fizz lucky 14 fizzbuzz 16 17 fizz 19 buzz

2 Step 2: FizzBuzz (Requisito nuevo)

Añade una regla nueva:

- Si el número **contiene un 3**, debe imprimirse **"lucky"**
- Esta regla **sobrescribe cualquier comportamiento anterior** (prioridad máxima)

Ejemplo (1–20):

1 2 lucky 4 buzz fizz 7 8 fizz buzz 11 fizz lucky 14 fizzbuzz 16 17 fizz 19 buzz

Hasta ahora teníamos reglas “combinables”:

- múltiplo de 3 → fizz
- múltiplo de 5 → buzz
- múltiplo de 15 → fizzbuzz

Ahora aparece una regla **de prioridad**:

- contiene 3 → lucky (y **corta** el resto)

Esto introduce dos complejidades típicas:

- **prioridad / precedencia** entre reglas
- **cambios no lineales** (no es “añadir un if más”, es reordenar la lógica)

📌 *Aquí es donde los ifs empiezan a volverse difíciles de mantener.*

★ Slide — Step 2: Cómo se complican los ifs (ejemplo realista) - Parte 1

📌 Observación:

- ya no hay un simple if/else lineal
- aparece una rama dominante (override)
- el método empieza a “mezclar” prioridad + composición de texto

```
public string ComputationOutputIterationFizzBuzz(int iteration)
{
    bool canBeMultipliedByFive = CanBeMultipliedBy(iteration, 5);
    bool canBeMultipliedByThree = CanBeMultipliedBy(iteration, 3);
    bool containsNumberThree = ContainsTheNumber(iteration, 3);
    string sequenceFizzBuzz = "";

    if (containsNumberThree)
    {
        sequenceFizzBuzz += "Lucky";
    }
    else
    {
        if (canBeMultipliedByThree) sequenceFizzBuzz += "Fizz";
        if (canBeMultipliedByFive) sequenceFizzBuzz += "Buzz";

        if (string.IsNullOrEmpty(sequenceFizzBuzz))
            sequenceFizzBuzz = iteration.ToString();
    }

    return sequenceFizzBuzz;
}
```

🔧 Slide — Step 2: Mini-implementación clara (prioridad + composición) - Parte 2

Esta versión es “limpia” para Step 2 (sin patrón aún), pero deja claro el flujo:

```
public static string Render(int n)
{
    if (n.ToString().Contains("3"))
        return "lucky";

    var result = "";
    if (n % 3 == 0) result += "fizz";
    if (n % 5 == 0) result += "buzz";

    return string.IsNullOrEmpty(result) ? n.ToString() : result;
}
```


Enunciado – Step 3

En los pasos anteriores, la lógica de **FizzBuzz** ha ido creciendo mediante condiciones (if/else) dentro de un único método. En este paso, el objetivo es **refactorizar** la solución aplicando el **Machine Rule Pattern**, de forma que:

Objetivos

- Cada regla de negocio esté **encapsulada** en una clase independiente.
- El algoritmo principal **no contenga lógica condicional compleja**.
- Las reglas se evalúen en **orden**, y la primera que aplique devuelva el resultado.
- El sistema sea **abierto a extensión y cerrado a modificación** (OCP – SOLID).

Reglas de negocio actuales

Para cada número de la secuencia:

1. Si el número **contiene el dígito 3**, devolver "Lucky".
2. Si el número es divisible por **3**, devolver "Fizz".
3. Si el número es divisible por **5**, devolver "Buzz".
4. Si es divisible por **3 y 5**, devolver "FizzBuzz".
5. Si no se cumple ninguna regla anterior, devolver el número como texto.

Regla como contrato (IRule)

Cada regla responde a dos preguntas:

1. ¿Aplica a este número?
2. ¿Qué valor genera?

```
namespace FizzBuzzStep4.Rule
{
    public interface IRule
    {
        bool Check(int number);
        string GenerateValue();
    }
}
```

 *Esto estandariza las reglas: el motor ya no depende de ifs.*

Reglas simples: Fizz y Buzz

Cada regla es una clase pequeña con una única responsabilidad.

```
namespace FizzBuzzStep4.Rule
{
    class FizzRule : IRule
    {
        const string VALUE = "Fizz";
        public bool Check(int number) => (number % 3 == 0);
        public string GenerateValue() => VALUE;
    }

    class BuzzRule : IRule
    {
        const string VALUE = "Buzz";
        public bool Check(int number) => (number % 5 == 0);
        public string GenerateValue() => VALUE;
    }
}
```

 *Aquí se ve el beneficio: cada regla es trivial de testear.*

Regla compuesta: FizzBuzz (15)

La regla compuesta evita la ambigüedad “fizz + buzz” si queremos un valor único.

```
namespace FizzBuzzStep4.Rule
{
    class FizzBuzzRule : IRule
    {
        const string VALUE = "FizzBuzz";
        public bool Check(int number) => (number % 15 == 0);
        public string GenerateValue() => VALUE;
    }
}
```

 Según el motor, puedes decidir si FizzBuzz reemplaza o concatena.

Regla con prioridad: Lucky (override)

Lucky introduce prioridad: cuando aplica, **corta** el resto.

```
namespace FizzBuzzStep4.Rule
{
    class LuckyRule : IRule
    {
        const string VALUE = "Lucky";
        public bool Check(int number) => number.ToString().Contains("3");
        public string GenerateValue() => VALUE;
    }
}
```

 *La prioridad se implementa en el motor, no en la regla.*

⚙️ Contrato del motor (IRuleMachine)

El motor debe poder:

- registrar reglas
- evaluar un número
- (opcional) generar reporte

```
using System.Collections.Generic;
using FizzBuzzStep4.Rule;

namespace FizzBuzzStep4.RuleMachine
{
    public interface IRuleMachine
    {
        void AddRule(IRule rule);
        string GenerateValueFizzBuzz(int number);
        void BuildReportFizzBuzz();
        Dictionary<string, int> GetReportFizzBuzz();
    }
}
```

📌 Se separa la “definición de reglas” de su “ejecución”.

⚙️ Contrato del motor (IRuleMachine)

El motor debe poder:

- registrar reglas
- evaluar un número
- (opcional) generar reporte

```
using System.Collections.Generic;
using FizzBuzzStep4.Rule;

namespace FizzBuzzStep4.RuleMachine
{
    public interface IRuleMachine
    {
        void AddRule(IRule rule);
        string GenerateValueFizzBuzz(int number);
        void BuildReportFizzBuzz();
        Dictionary<string, int> GetReportFizzBuzz();
    }
}
```

📌 Se separa la “definición de reglas” de su “ejecución”.



Motor: evaluación ordenada + corte por Lucky

El motor recorre las reglas en orden y se queda con el valor generado.
Si es Lucky → **break** para aplicar override.

```
private void IncreaseRule(string rule)
{
    if (reportFizzBuzz.ContainsKey(rule))
        reportFizzBuzz[rule]++;
    else
        reportFizzBuzz["Integer"]++;
}

public Dictionary<string, int> GetReportFizzBuzz() => reportFizzBuzz;
```



El orden en rules es tu “política de prioridad”.

```
class FizzBuzzRuleMachine : IRuleMachine
{
    private List<IRule> rules = new List<IRule>();
    private Dictionary<string, int> reportFizzBuzz = new Dictionary<string, int>();

    public void AddRule(IRule rule) => rules.Add(rule);

    public void BuildReportFizzBuzz()
    {
        reportFizzBuzz["Fizz"] = 0;
        reportFizzBuzz["Buzz"] = 0;
        reportFizzBuzz["FizzBuzz"] = 0;
        reportFizzBuzz["Lucky"] = 0;
        reportFizzBuzz["Integer"] = 0;
    }

    public string GenerateValueFizzBuzz(int number)
    {
        string ruleName = number.ToString();

        foreach (var rule in rules)
        {
            if (rule.Check(number))
            {
                ruleName = rule.GenerateValue();
                if (ruleName == "Lucky")
                    break; // prioridad absoluta
            }
        }

        IncreaseRule(ruleName);
        return ruleName;
    }
}
```




Motor: evaluación ordenada + corte por Lucky

El motor recorre las reglas en orden y se queda con el valor generado.
Si es Lucky → **break** para aplicar override.

```
private void IncreaseRule(string rule)
{
    if (reportFizzBuzz.ContainsKey(rule))
        reportFizzBuzz[rule]++;
    else
        reportFizzBuzz["Integer"]++;
}

public Dictionary<string, int> GetReportFizzBuzz() => reportFizzBuzz;
```



El orden en rules es tu “política de prioridad”.

```
class FizzBuzzRuleMachine : IRuleMachine
{
    private List<IRule> rules = new List<IRule>();
    private Dictionary<string, int> reportFizzBuzz = new Dictionary<string, int>();

    public void AddRule(IRule rule) => rules.Add(rule);

    public void BuildReportFizzBuzz()
    {
        reportFizzBuzz["Fizz"] = 0;
        reportFizzBuzz["Buzz"] = 0;
        reportFizzBuzz["FizzBuzz"] = 0;
        reportFizzBuzz["Lucky"] = 0;
        reportFizzBuzz["Integer"] = 0;
    }

    public string GenerateValueFizzBuzz(int number)
    {
        string ruleName = number.ToString();

        foreach (var rule in rules)
        {
            if (rule.Check(number))
            {
                ruleName = rule.GenerateValue();
                if (ruleName == "Lucky")
                    break; // prioridad absoluta
            }
        }

        IncreaseRule(ruleName);
        return ruleName;
    }
}
```



Orquestador: configura el motor (qué reglas y en qué orden)

Aquí se decide la “estrategia” de evaluación: reglas y prioridad.

```
class FizzBuzz
{
    private const int INITIAL = 1;
    private const int TOTAL = 100;

    private IRuleMachine ruleMachine;

    public FizzBuzz()
    {
        ruleMachine = new FizzBuzzRuleMachine();
        ruleMachine.AddRule(new LuckyRule()); // prioridad
        ruleMachine.AddRule(new BuzzRule());
        ruleMachine.AddRule(new FizzRule());
        ruleMachine.AddRule(new FizzBuzzRule()); // (según político)
    }

    public void BuildReportFizzBuzz() => ruleMachine.BuildReportFizzBuzz();

    public void SequenceFizzBuzz()
    {
        for (int i = INITIAL; i <= TOTAL; i++)
        {
            string result = ruleMachine.GenerateValueFizzBuzz(i);
            Console.WriteLine(result + " ");
        }
    }

    public void PrintReportFizzBuzz()
    {
        Dictionary<string, int> report = ruleMachine.GetReportFizzBuzz();
        foreach (var item in report)
        {
            Console.WriteLine(item.Key + " : " + item.Value);
        }
    }
}
```



La configuración del motor es lo que cambia cuando cambian requisitos, no el motor.

▶ Program.cs (Punto de entrada / ejecución)

📌 Observación:

- BuildRuleMachine() configura reglas y prioridad (política)
- BuildReportFizzBuzz() inicializa contadores (métrica)
- SequenceFizzBuzz() ejecuta el rango y genera salidas
- PrintReportFizzBuzz() imprime resumen final

```
using FizzBuzzStep4.RuleMachine;
using System;
using System.Collections.Generic;

namespace FizzBuzzStep4
{
    class Program
    {
        static void Main(string[] args)
        {
            FizzBuzz fizzBuzz = new FizzBuzz();

            fizzBuzz.BuildRuleMachine();
            fizzBuzz.BuildReportFizzBuzz();
            fizzBuzz.SequenceFizzBuzz();
            fizzBuzz.PrintReportFizzBuzz();

            Console.WriteLine("Press ENTER to close console.....");
            Console.ReadLine();
        }
    }
}
```